



Python p/ Processamento de Dados

Etapa 7

Professor(a):

E-mail: marcelo.hama@prof.infnet.edu.br

Como Tratar Bugs

Nesta aula abordaremos os principais mecanismos de tratamento de erros e debug em Python, tópicos essenciais para escrever código robusto, confiável e profissional.

Conteúdo da Aula:

- Parte 1 — Try-Except: captura e tratamento de exceções em tempo de execução;
- Parte 2 — Try-Except-Else: separação entre fluxo de sucesso e fluxo de erro;
- Parte 3 — Diretiva Pass: silenciar exceções esperadas de forma controlada;
- Parte 4 — Debug com IDE e PDB: inspecionar e corrigir código com precisão.

Referências:

- Python Crash Course (Matthes, 3a ed.) — Cap. 10;
- Introducing Python (Lubanovic, 2a ed.) — Cap. 19.

Como Tratar Bugs

PARTE 1

Try-Except Para Tratar Exceções

Try-Except Para Tratar Exceções

O que é uma Exceção?

Uma exceção é um evento anormal que ocorre durante a execução de um programa e interrompe seu fluxo normal. O Python representa cada tipo de erro como um objeto de uma classe de exceção específica.

Como o Python lida com exceções sem tratamento:

- O interpretador para a execução imediatamente ao encontrar o erro;
- Exibe um traceback (rastreamento de pilha) com o tipo e a mensagem do erro;
- Indica a linha exata onde o problema ocorreu;
- O programa encerra com código de saída diferente de zero.

```
1 10/0
```

❗ Execution error

`ZeroDivisionError`: division by zero

Try-Except Para Tratar Exceções

Hierarquia de Exceções em Python

Todas as exceções herdam de `BaseException`. As mais comuns herdam de `Exception`:

BaseException

- +-- SystemExit # sys.exit()*
- +-- KeyboardInterrupt # Ctrl+C*
- +-- Exception # base de todas as comuns*
 - +-- ValueError # valor inválido*
 - +-- TypeError # tipo incorreto*
 - +-- OSError # erros de I/O*
 - | +-- FileNotFoundError*
 - | +-- PermissionError*
 - +-- ArithmeticError*
 - | +-- ZeroDivisionError*
 - +-- LookupError*
 - +-- KeyError / IndexError*

Capturar `Exception` cobre a maioria dos erros do programa;

Capturar `BaseException` inclui também `SystemExit` e `KeyboardInterrupt` — use com cautela.

Try-Except Para Tratar Exceções

Sintaxe Básica do try-except

O bloco try envolve o que pode falhar. O bloco except define o que fazer se a falha ocorrer:

try:

```
# código que pode gerar uma exceção  
# sintaxe: resultado = operacao_arriscada()  
resultado = 10/0
```

sintaxe: except NomeDaExcecao:

except ZeroDivisionError:

```
# código executado se a exceção ocorrer  
# print('Algo deu errado.')  
print('Houve uma divisao por zero!')
```

Regras fundamentais:

- O bloco except só é executado se uma exceção do tipo declarado ocorrer;
- Se nenhuma exceção ocorrer, o except é completamente ignorado;
- Após o except, o programa continua normalmente;
- Um try pode ter múltiplos blocos except para tipos diferentes.

Try-Except Para Tratar Exceções

Exemplo Prático: Leitura de Arquivo

Sem tratamento, o programa para se o arquivo não existir. Com try-except, temos o controle:

Sem tratamento (frágil):

```
# trava se dados.txt não existir  
with open('dados.txt', 'r') as f:  
    conteudo = f.read()
```

Com try-except (robusto):

```
try:  
    with open('dados.txt', 'r',  
encoding='utf-8') as f:  
        conteudo = f.read()  
        print(f'Lidos {len(conteudo)}  
caracteres.')  
except FileNotFoundError:  
    print('Arquivo dados.txt não  
encontrado.')  
    conteudo = ''
```

Try-Except Para Tratar Exceções

Capturando Múltiplos Tipos de Exceção

Podemos encadear vários blocos except para tratar cada tipo de erro de forma específica:

try:

```
entrada = input('Digite um número: ')
```

```
numero = int(entrada)
```

```
resultado = 100 / numero
```

```
print(f'Resultado: {resultado:.2f}')
```

except ValueError:

```
print('Erro: entrada não é um número válido.')
```

except ZeroDivisionError:

```
print('Erro: divisão por zero não é permitida.')
```

Importante:

- A ordem dos except importa: exceções mais específicas primeiro;
- FileNotFoundError antes de OSError (pois a primeira é subclasse da segunda);
- Nunca coloque Exception como primeiro except — capturaria tudo.

Try-Except Para Tratar Exceções

Capturando Múltiplos Tipos em um Único except

Quando o mesmo tratamento serve para mais de um tipo de erro, use uma tupla:

```
try:
    desconto = float(dados['desconto'])
    preco_final = float(dados['preco']) * (1 - desconto)
except (KeyError, ValueError, TypeError) as e:
    print(f'Dados inválidos: {e}')
    preco_final = 0.0
```

Uso do parâmetro as:

- **as e** captura o objeto da exceção, permitindo inspecionar sua mensagem;
- O objeto tem atributo `args` com os argumentos passados ao construtor;
- `str(e)` retorna a mensagem legível da exceção.

Try-Except Para Tratar Exceções

Exceções Comuns em Python — Referência Rápida

- ValueError — valor inválido para o tipo: `int('abc')`, `float('--')`
- TypeError — tipo incorreto: `'texto' + 5`, `len(42)`
- FileNotFoundError — arquivo ausente: `open('nao_existe.txt')`
- PermissionError — sem permissão de acesso ao arquivo ou diretório
- ZeroDivisionError — divisão por zero: `10 / 0`, `10 % 0`
- KeyError — chave inexistente em dict: `d['chave_ausente']`
- IndexError — índice fora do intervalo: `lista[100]` com 3 elementos
- AttributeError — atributo inexistente: `None.split()`
- NameError — variável não definida: `print(variavel_inexistente)`
- ImportError / ModuleNotFoundError — módulo não encontrado
- StopIteration — iterador esgotado (raramente capturado diretamente)
- RecursionError — limite de recursão excedido

Try-Except Para Tratar Exceções

O Bloco *finally*: Execução Garantida

O *finally* executa sempre, independentemente do resultado do *try* — com ou sem exceção:

try:

```
conexao = abrir_conexao_banco()
dados   = conexao.executar_query('SELECT ...')
```

except DatabaseError as e:

```
print(f'Erro no banco: {e}')
dados = []
```

finally:

```
conexao.fechar() # garante fechamento sempre
print('Conexão encerrada.')
```

Casos de uso do *finally*:

- Fechar arquivos, sockets ou conexões de banco de dados;
- Liberar locks ou recursos de sistema;
- Registrar o fim de uma operação em log;
- Restaurar estado global ou limpar variáveis temporárias.

Try-Except Para Tratar Exceções

Levantando Exceções com raise

Podemos também lançar exceções para comunicar condições inválidas ao chamador:

```
def calcular_raiz(numero):  
    if not isinstance(numero, (int, float)):  
        raise TypeError('Esperado int ou float.')  
    if numero < 0:  
        raise ValueError(f'{numero} é negativo. Raiz indefinida.')  
    return numero ** 0.5
```

```
try:  
    print(calcular_raiz(-4))  
except ValueError as e:  
    print(f'Erro de valor: {e}')
```

raise sem argumento — relança a exceção atual:

```
except Exception as e:  
    logging.error(e)  
    raise # propaga para o chamador
```

Try-Except Para Tratar Exceções

Exceções Personalizadas

Para domínios específicos, crie suas próprias classes de exceção herdando de Exception, tornando o código mais expressivo e fácil de depurar:

```
class SaldoInsuficienteError(Exception):  
    def __init__(self, saldo, valor):  
        self.saldo = saldo  
        self.valor = valor  
        super().__init__(  
            f'Saldo {saldo:.2f} insuficiente')
```

```
def sacar(saldo, valor):  
    if saldo < valor:  
        raise SaldoInsuficienteError(saldo, valor)  
    saldo -= valor
```

```
try:  
    sacar(500, 1000.00)  
except SaldoInsuficienteError as e:  
    print(e)    # Saldo 250.00 insuficiente
```

Try-Except Para Tratar Exceções

Boas Práticas com try-except

Faça:

- Capture o tipo mais específico possível (FileNotFoundError, não OSError);
- Use as e para registrar a mensagem do erro em logs;
- Forneça um valor padrão seguro no except quando fizer sentido;
- Use finally para liberar recursos independentemente do resultado;
- Documente quais exceções uma função pode lançar.

Evite:

- except: vazio — captura até SystemExit e KeyboardInterrupt;
- except Exception: pass — silencia todos os erros sem registrar nada;
- Blocos try muito longos — minimize o código sujeito a falha;
- Capturar exceções que você não sabe como tratar adequadamente;
- Usar exceções para controle de fluxo normal do programa.

Try-Except Para Tratar Exceções

Tratamento de Exceções em Funções com JSON

Exemplo real e completo: leitura e validação de arquivo de configuração JSON. Padrão recomendado: cada tipo de erro com ação específica e valor de retorno seguro.

```
import json

def carregar_config(caminho='config.json'):
    try:
        with open(caminho, 'r', encoding='utf-8') as f:
            config = json.load(f)
    except FileNotFoundError:
        print(f'[AVISO] {caminho} não encontrado. Usando padrões.')
        return {}
    except json.JSONDecodeError as e:
        print(f'[ERRO] JSON inválido: {e}')
        return {}
    except PermissionError:
        print(f'[ERRO] Sem permissão para ler o arquivo.')
        return {}
    else:
        print(f'Config carregada: {len(config)} chaves.')
        return config
```

Como Tratar Bugs

PARTE 2

Bloco Try-Except-Else para Tratar Exceções

Bloco Try-Except-Else para Tratar Exceções

Estrutura Completa: try-except-else-finally

Python oferece quatro blocos complementares para tratamento de exceções:

try:

código que pode gerar exceção

except TipoA:

tratamento específico para TipoA

except TipoB as e:

tratamento específico para TipoB

else:

executado SOMENTE se try terminou sem exceção

finally:

executado SEMPRE, com ou sem exceção

Regra de execução:

- try com sucesso: try → else → finally;
- try com erro capturado: try (parcial) → except → finally;
- try com erro não capturado: try (parcial) → finally → propaga o erro.

Bloco Try-Except-Else para Tratar Exceções

Por Que Existe o Bloco *else*?

O *else* resolve uma ambiguidade: código no *try* é protegido mesmo quando não deveria ser, tornando a intenção explícita: um código que só faz sentido se o *try* teve sucesso.

Problema — `processar()` também é capturado por `IOError`:

```
try:
    # pode falhar
    f = open('dados.txt')
    # capturado por engano
    processar(f.read())
except IOError:
    print('Erro de I/O')
```

Solução com *else* — cada bloco tem responsabilidade clara:

```
try:
    # apenas isso é protegido
    f = open('dados.txt')
except IOError:
    print('Erro ao abrir arquivo')
else:
    # executado somente se abriu
    processar(f.read())
```

Bloco Try-Except-Else para Tratar Exceções

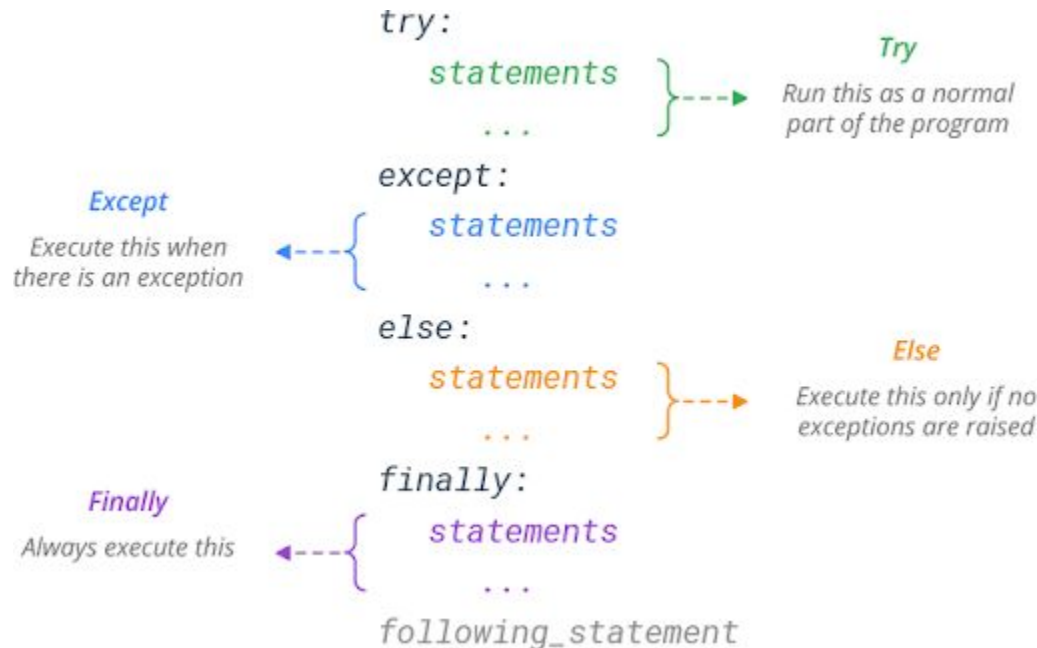
Exemplo completo com *try-except-else-finally*. Note que *finally* executa em ambos os casos, sempre após *except* ou *else*.

```
def dividir(a, b):  
    try:  
        resultado = a / b  
    except ZeroDivisionError:  
        print('Divisão por zero.')  
        resultado = None  
    else:  
        print(f'Resultado: {resultado:.4f}')  
    finally:  
        print(f'Tentativa: {a} / {b}')  
    return resultado  
  
dividir(10, 3) # Resultado: 3.3333  
dividir(10, 0) # Divisão por zero.
```

Bloco Try-Except-Else para Tratar Exceções

Fluxo de Execução — Diagrama

Entender o fluxo evita surpresas com código executando inesperadamente:



Bloco Try-Except-Else para Tratar Exceções

Leitura de JSON com Estrutura Completa

```
import json
try:
    with open('pedidos.json', 'r', encoding='utf-8') as f:
        pedidos = json.load(f)
except FileNotFoundError:
    print('[ERRO] Arquivo pedidos.json não encontrado.')
    pedidos = []
except json.JSONDecodeError as e:
    print(f'[ERRO] JSON malformatado na linha {e.lineno}.')
    pedidos = []
except PermissionError:
    print('[ERRO] Sem permissão para acessar o arquivo.')
    pedidos = []
else:
    total = sum(p['valor'] for p in pedidos)
    print(f'{len(pedidos)} pedidos | Total: R$ {total:.2f}')
finally:
    print('Tentativa de leitura concluída.')
```

Bloco Try-Except-Else para Tratar Exceções

else com Processamento de Dados

O *else* é especialmente útil quando o processamento principal depende do sucesso de uma operação de I/O:

```
import csv

def processar_csv(caminho):
    registros = []
    try:
        f = open(caminho, newline='', encoding='utf-8')
    except FileNotFoundError:
        print(f'Arquivo {caminho} não encontrado.')
    except PermissionError:
        print('Sem permissão de leitura.')
    else:
        with f:
            reader = csv.DictReader(f)
            registros = [row for row in reader
                        if float(row['valor']) > 0]
        print(f'{len(registros)} registros válidos processados.')
    finally:
        return registros
```

Bloco Try-Except-Else para Tratar Exceções

Aninhamento de try-except

Em situações complexas, try-except pode ser aninhado para granularidade maior. Em geral, mais de dois níveis geralmente indica necessidade de refatoração.

```
def salvar_resultado(dados, caminho):  
    try:  
        resultado = processar(dados)  
    except ValueError as e:  
        print(f'Dados inválidos: {e}')  
        return False  
    else:  
        try:  
            with open(caminho, 'w') as f:  
                json.dump(resultado, f, indent=2)  
        except IOError as e:  
            print(f'Erro ao salvar: {e}')  
            return False  
        else:  
            print(f'Salvo em {caminho}')  
            return True
```

Bloco Try-Except-Else para Tratar Exceções

Transformando Exceções com *raise* Inside *except*

É possível capturar uma exceção e relançar um tipo diferente, mais adequado ao contexto:

```
class ConfiguracaoInvalidaError(Exception):
    pass

def carregar_porta(config):
    try:
        porta = int(config['porta'])
    except KeyError:
        raise ConfiguracaoInvalidaError(
            'Chave porta ausente na configuração.')
    except ValueError:
        raise ConfiguracaoInvalidaError(
            f"Porta inválida: {config['porta']!r}")
    else:
        if not (1024 <= porta <= 65535):
            raise ConfiguracaoInvalidaError(
                f'Porta {porta} fora do intervalo permitido.')
        return porta
```


Bloco Try-Except-Else para Tratar Exceções

Uso com *Context Managers* (*with*)

O *with* é a forma preferida de gerenciar recursos em Python, equivalente a *try-finally*. No exemplo, o *with* garante o fechamento do arquivo mesmo se *json.load* lançar uma exceção.

Equivalência:

```
# Com with (preferido):  
with open('arq.txt') as f:  
    dados = f.read()  
  
# Equivalente manual:  
f = open('arq.txt')  
try:  
    dados = f.read()  
finally:  
    f.close()
```

Combinando with e except:

```
try:  
    with open('config.json') as f:  
        config = json.load(f)  
except (FileNotFoundError, json.JSONDecodeError) as e:  
    print(f'Erro: {e}')  
    config = {}
```

Bloco Try-Except-Else para Tratar Exceções

Validação de Entrada com *try-except-else*

Padrão comum para leitura interativa com validação robusta:

```
def ler_inteiro(prompt, minimo=None, maximo=None):  
    while True:  
        try:  
            valor = int(input(prompt))  
        except ValueError:  
            print('  Entrada inválida. Digite um número inteiro.')  
        else:  
            if minimo is not None and valor < minimo:  
                print(f'  Valor deve ser >= {minimo}.')  
            elif maximo is not None and valor > maximo:  
                print(f'  Valor deve ser <= {maximo}.')  
            else:  
                return valor
```

```
idade = ler_inteiro('Idade: ', minimo=0, maximo=150)
```

Bloco Try-Except-Else para Tratar Exceções

Boas Práticas com *try-except-else-finally*

Faça:

- *try*: apenas o mínimo necessário (a operação que pode falhar);
- *except*: um por tipo de erro, com ação específica para cada;
- *else*: processamento que depende do sucesso do *try*;
- *finally*: limpeza e liberação de recursos.

Evite:

- Colocar processamento longo no *try* (aumenta o escopo de captura);
- Misturar lógica de negócio com tratamento de erro no mesmo bloco;
- Usar *finally* como substituto para *else* (semânticas diferentes);
- Suprimir exceções sem registrar (use *logging.exception()* no *except*).

Como Tratar Bugs

PARTE 3

Diretiva Pass Para Silenciar Exceções

Diretiva Pass Para Silenciar Exceções

O que é a instrução pass?

O *pass* é uma instrução nula do Python, que ocupa um lugar sintaticamente obrigatório sem executar nenhuma ação:

```
try:
    resultado = int('abc')
except ValueError:
    pass    # bloco exigido
```

```
### Outros usos de pass além de except:
# classe vazia
class MinhaExcecao(Exception): pass
# implementar depois
def funcao_placeholder(): pass
# condição sem ação
if debug_mode: pass
```

Por que é necessária?

- O Python exige ao menos uma instrução em qualquer bloco (*if*, *for*, *def*, *class*, *except*);
- Um bloco vazio é erro de sintaxe e o *pass* resolve isso explicitamente;
- Transmite intenção ao leitor: 'este caso foi considerado e ignorado propositalmente'.

Diretiva Pass Para Silenciar Exceções

***pass* vs silêncio implícito**

Em outras linguagens, um bloco catch vazio não precisa de instrução. Em Python, *pass* é explícito e comunica intenção. A obrigatoriedade de *pass* é um recurso de design, que força o programador a ser explícito sobre a supressão.

Java — bloco vazio funciona:

```
try { int x = Integer.parseInt("abc"); }  
catch (NumberFormatException e) { } // sem instrução
```

Python — *pass* é obrigatório e comunicativo:

```
try:  
    x = int('abc')  
except ValueError:  
    # explícito: sei do erro, escolhi ignorar  
    pass
```

Diretiva Pass Para Silenciar Exceções

Casos de Uso: Quando pass é Legítimo usar o pass.

1. Remoção preventiva de arquivo (pode ou não existir):

```
import os
try:
    os.remove('cache_temp.json')
except FileNotFoundError:
    pass # normal - arquivo pode não ter sido criado
```

2. Carregamento opcional de configuração local:

```
config = carregar_config_padrao()
try:
    config.update(carregar_config_local())
except FileNotFoundError:
    pass # config local é opcional, sem problema
```

3. Ignorar erros de formatação em processamento em lote:

```
for linha in linhas:
    try:
        registros.append(parsear(linha))
    except (ValueError, KeyError):
        pass # linha malformada - ignora e segue
```

Diretiva Pass Para Silenciar Exceções

pass em Processamento em Lote

Em pipelines de dados, erros em registros individuais não devem interromper o processamento:

```
def processar_lote(registros):  
    resultados = []  
    erros = 0  
    for i, registro in enumerate(registros):  
        try:  
            resultado = transformar(registro)  
            resultados.append(resultado)  
        except (ValueError, TypeError):  
            erros += 1  
            pass # registra contagem, ignora o registro  
    print(f'Processados: {len(resultados)} | Erros: {erros}')  
    return resultados
```

Substitua pass por logging para rastreabilidade sem interrupção

```
except (ValueError, TypeError) as e:  
    logging.debug(f'Registro {i} ignorado: {e}')
```


Diretiva Pass Para Silenciar Exceções

***pass* em Prototipagem e Desenvolvimento**

Durante o desenvolvimento, *pass* pode marcar código que será implementado depois. Use comentários TODO junto ao *pass* para sinalizar implementação pendente.

Classes e funções placeholder:

```
class ProcessadorDados:
    def carregar(self): pass      # TODO
    def transformar(self): pass  # TODO
    def exportar(self): pass     # TODO
```

Tratamento de erro planejado mas não implementado:

```
try:
    resultado = chamada_api()
except ConnectionError:
    pass # TODO: implementar retry com backoff
except TimeoutError:
    pass # TODO: notificar monitoramento
```

Diretiva Pass Para Silenciar Exceções

Comparação entre *pass* vs *continue* vs *break*

As três instruções controlam fluxo mas com propósitos bem distintos. Em *except*, apenas *pass* faz sentido, enquanto que *continue* e *break* só existem dentro de loops.

pass — não faz nada, apenas ocupa o bloco:

```
for item in lista:
    if condicao(item):
        pass          # processa igualmente - sem efeito
    processar(item)   # sempre executado
```

continue — pula para a próxima iteração:

```
for item in lista:
    if item is None:
        continue      # pula None, vai ao próximo
    processar(item)   # executado apenas se não None
```

break — interrompe o loop inteiramente:

```
for item in lista:
    if item == SENTINELA:
        break          # para o loop
    processar(item)
```

Diretiva Pass Para Silenciar Exceções

Quando NÃO Usar o *pass*: Armadilhas de Uso

Perigo 1 — Suprimir erros reais:

```
try:
    gravar_em_banco(dados)
except Exception:
    # PERIGO: falha silenciosa
    pass
```

Perigo 2 — Em `except Exception` genérico:

```
try:
    operacao_complexa()
except Exception: # captura TUDO
    pass          # e ignora TUDO - inaceitável
```

Perigo 3 — Escondendo bug:

```
try:
    # bug: retorna None
    total = soma(valores)
# captura None + número
except TypeError:
    # bug nunca é detectado
    pass
```

Diretiva Pass Para Silenciar Exceções

Alternativas ao pass: Ações Mínimas

Em vez de *pass* puro, considere ações leves que preservam rastreabilidade. Use *pass* apenas quando silêncio completo for explicitamente a intenção correta.

Logging — registra sem interromper:

```
import logging
except FileNotFoundError as e:
    logging.debug(f'Arquivo opcional ausente: {e}')
```

Valor padrão — fornece fallback seguro:

```
except (KeyError, ValueError):
    preco = 0.0    # mais explícito que pass
```

Contar erros — monitora frequência:

```
except Exception:
    erros_totais += 1
```

warnings — avisa sem parar:

```
import warnings
except DeprecationWarning:
    warnings.warn('Recurso obsoleto', stacklevel=2)
```

Diretiva Pass Para Silenciar Exceções

Supressão Contextual com *contextlib.suppress*

Python 3 oferece uma alternativa elegante ao *try/except/pass* para suprimir exceções específicas:

Com *try-except-pass* (verboso):

```
try:
```

```
    os.remove('temp.json')
```

```
except FileNotFoundError:
```

```
    pass
```

Com *contextlib.suppress* (mais expressivo):

```
from contextlib import suppress
```

```
with suppress(FileNotFoundError):
```

```
    os.remove('temp.json')
```

Suprimindo múltiplos tipos:

```
# suppress é equivalente a except + pass, mas mais legível e explícito.
```

```
with suppress(FileNotFoundError, PermissionError):
```

```
    os.remove('arquivo_protegido.txt')
```

Diretiva Pass Para Silenciar Exceções

Resumo: *pass* em Contextos de Exceção

Use *pass* quando:

- A exceção é completamente esperada e não indica problema;
- A operação é opcional e a ausência é um estado normal;
- Você está fazendo prototipagem e implementará depois (com TODO);
- O erro em um registro individual não deve parar o processamento do lote.

Prefira alternativas quando:

- O erro precisa ser registrado para análise futura (use logging);
- É necessário fornecer um valor de fallback (forneça explicitamente);
- A exceção indica estado inconsistente (trate ou propague);
- Você está capturando Exception ou BaseException (nunca silencie com *pass*).

pass comunica “*sei desse erro e escolhi ignorá-lo*”. Use apenas quando isso for verdade.

Como Tratar Bugs

PARTE 4

Debug de Códigos Python com IDE/PDB

Debug de Códigos Python com IDE/PDB

O que é Debug e por que é essencial?

Debug é o processo sistemático de encontrar e corrigir erros em código. Ao contrário de ler o código estaticamente, o debug permite inspecionar o estado real durante a execução.

Tipos de erros em Python:

- Erros de sintaxe — detectados antes de executar (*SyntaxError*, *IndentationError*);
- Erros de execução (runtime) — ocorrem durante a execução (*TypeError*, *ValueError*);
- Erros de lógica — código executa sem erros, mas produz resultado incorreto.

Ferramentas disponíveis:

- `print()` — simples, sem instalação, mas polui o código e é limitado;
- PDB — depurador nativo, via terminal, sem dependências externas;
- VS Code + extensão Python — depurador visual, gratuito e extensível.

Debug de Códigos Python com IDE/PDB

Lendo o Traceback

O traceback é o mapa do erro. Lê-se de baixo para cima:

Traceback (most recent call last):

File "app.py", line 15, in main

resultado = processar(dados)

File "app.py", line 8, in processar

total = sum(item['valor'] for item in dados)

File "app.py", line 8, in <genexpr>

total = sum(item['valor'] for item in dados)

KeyError: 'valor'

1. Última linha: tipo (KeyError) e detalhe ('valor' — chave ausente);
2. Linha acima: código exato que falhou e o arquivo/linha;
3. Linhas acima: caminho de chamadas até chegar ao erro;
4. Primeira linha: ponto de entrada (main, linha 15).

Diagnóstico: dicionário item não possui a chave 'valor'. Verifique os dados de entrada.

Debug de Códigos Python com IDE/PDB

Debug com *print()*

O *print()* é a ferramenta de debug básica. Útil para localizar erros, mas deve ser removido depois:

```
def calcular_media(notas):  
    print(f'[DEBUG] notas recebidas: {notas}')      # tipo?  
    print(f'[DEBUG] tipo: {type(notas)}')          # lista?  
    total = sum(notas)  
    print(f'[DEBUG] total: {total}')                # correto?  
    media = total / len(notas)  
    print(f'[DEBUG] media: {media}')                # esperado?  
    return media
```

Limitações do *print()*:

- Polui o código e precisa ser removido manualmente;
- Não permite inspecionar estado após uma exceção;
- Não permite modificar variáveis durante a execução;

Debug de Códigos Python com IDE/PDB

PDB — Introdução ao Depurador Nativo

O PDB (Python Debugger) pausa a execução em um ponto e abre um prompt para inspeção:

Ativação clássica com `pdb.set_trace()`:

```
import pdb
def calcular(x, y):
    pdb.set_trace() # pausa aqui, abre (Pdb)
    return x / y
print(calcular(20, 10))
```

Forma moderna — Python 3.7+:

```
def calcular(x, y):
    breakpoint() # equivalente ao set_trace
    return x / y
print(calcular(20, 10))
```

O `breakpoint()` usa `PYTHONBREAKPOINT` env var para configurar o depurador. Defina `PYTHONBREAKPOINT=0` para desabilitar todos os breakpoints sem alterar código.

Debug de Códigos Python com IDE/PDB

Comandos Essenciais do PDB

- n (next) — executa a próxima linha sem entrar em funções chamadas;
- s (step) — entra dentro da função chamada na linha atual;
- r (return) — continua até o return da função atual;
- c (continue) — continua até o próximo breakpoint ou fim do programa;
- q (quit) — encerra o depurador e o programa imediatamente;
- l (list) — exibe 11 linhas ao redor da posição atual;
- ll (longlist) — exibe a função completa atual;
- p <expr> — avalia e imprime o valor de uma expressão;
- pp <expr> — pretty-print (melhor para dicts e listas grandes);
- b <linha> — define breakpoint na linha especificada;
- b — lista todos os breakpoints definidos;
- cl <num> — remove o breakpoint de número num;
- w (where) — exibe a pilha de chamadas completa;
- u / d — move para cima/baixo na pilha de chamadas.

Debug de Códigos Python com IDE/PDB

Sessão PDB — Exemplo Prático

Debug passo a passo de uma função com erro de lógica:

```
import pdb
def media_ponderada(notas, pesos):
    pdb.set_trace()
    return sum(n*p for n,p in zip(notas,pesos)) / sum(pesos)
media_ponderada([8, 6, 9], [2, 3, 1])
```

(Pdb) p notas

[8, 6, 9]

(Pdb) p pesos

[2, 3, 1]

(Pdb) p list(zip(notas, pesos))

[(8, 2), (6, 3), (9, 1)]

*(Pdb) p sum(n*p for n,p in zip(notas,pesos))*

43

(Pdb) p sum(pesos)

6

(Pdb) n # executa o return

(Pdb) c # continua

7.166666666666667

Debug de Códigos Python com IDE/PDB

PDB Post-Mortem — Debug Após Exceção

O PDB pode ser acionado automaticamente quando uma exceção não capturada ocorre:

Via linha de comando:

```
$ python -m pdb meu_script.py
```

Post-mortem manual no código:

```
import pdb, traceback
try:
    funcao_com_bug()
except Exception:
    traceback.print_exc()
    pdb.post_mortem() # abre PDB no frame do erro
```

O Post-mortem permite inspecionar as variáveis no estado exato em que o erro ocorreu.

Debug de Códigos Python com IDE/PDB

Estratégias de Debug Eficiente

1. Reduza o problema ao mínimo reproduzível

Isolate o trecho que causa o erro. Quanto menor, mais rápido o diagnóstico.

2. Leia o traceback de baixo para cima

A última linha indica o tipo e a mensagem. As acima mostram o caminho.

3. Verifique tipos antes de valores

(Pdb) p type(dados) # dict ou list?

(Pdb) p len(dados) # vazio?

(Pdb) p dados.keys() # quais chaves existem?

4. Use breakpoints condicionais para loops grandes

Em loops de milhares de iterações, breakpoints incondicionais são lentos demais.

5. Combine try-except com debug

except Exception:

import pdb; pdb.set_trace() # inspeciona no exato momento do erro

6. Registre hipóteses e valide uma por vez

Debug é científico: formule uma hipótese, teste, confirme ou refute, avance.

Referências Bibliográficas

- Get Programming, Ana Bell, Manning Publications;
- Introducing Python, Bill Lubanovic, O'Reilly Media, Inc;
- Learn to Code by Solving Problems, Daniel Zingaro, No Starch Press queue;
- Python Crash Course, Eric Matthes, No Starch Press;
- Streamlining Your Research Laboratory with Python, Mark F. Russo, William Neil, Wiley;
- The Quick Python Book, Naomi Ceder, Manning Publications;
- <https://deepnote.com/docs/incoming-connections>;
- <https://ai.google.dev/gemini-api/docs>;
- <https://developers.openai.com/api/docs/libraries>.